N-gram Language Modeling

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

In this study, we explore linear interpolation smoothing in n-gram language models to enhance model performance on unseen data. With the goal of reducing perplexity, we generate smoothed probabilities using a weighted linear interpolation method that combines unigram, bigram, and trigram models. We test our method on training, development, and test datasets using a subset of the One Billion Word Language Modeling Benchmark. Through hyperparameter tuning with Optuna, we optimize the interpolation weights (λ_1, λ_2, λ_3), enabling effective exploration of different weight combinations. With the ideal weights of $\lambda_1 = 0.216$, $\lambda_2 = 0.63$, and $\lambda_3 = 0.144$, we obtain perplexities of 25.48 on the training set, 188.6 on the development set, and 307.92 on the test set. Further analysis on training data volume and <UNK> tokenization strategies provides insights into how these factors impact model generalization.

1 Introduction

1.1 Dataset Description

In this assignment, we use a dataset consisting of three data files (a subset of the One Billion Word Language Modeling Benchmark), where each line in each file is a whitespace-tokenized sentence:

- **1b_benchmark.train.tokens**: Data for training the language models.
- **1b_benchmark.dev.tokens**: Data for debugging and selecting the best hyperparameters.
- **1b_benchmark.test.tokens**: Data for evaluating the language models.

2 Methodology

2.1 Handling Out-of-Vocabulary (OOV) Words

We count word frequencies in the data. Words that appear fewer than a specified minimum count (min_count, set to 3 here) are replaced with the token <UNK>. From the plots, we can observe that in each dataset, the OOV words (represented by <UNK>) exceed those that are not <UNK>.

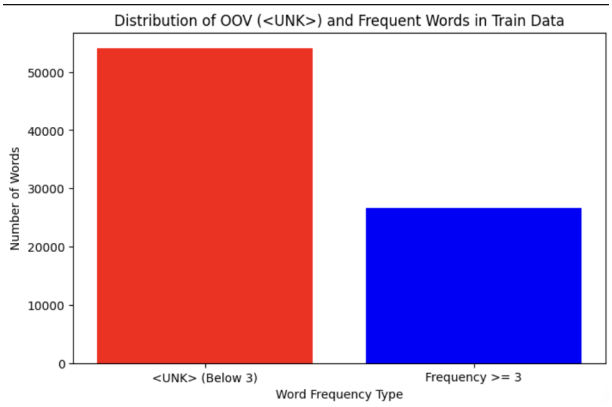

Figure 1: Distribution of OOV (<UNK>) and Frequent Words in Train Data

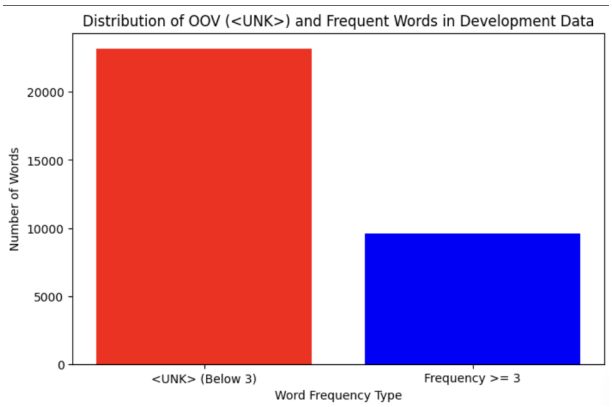

Figure 2: Distribution of OOV (<UNK>) and Frequent Words in Development Data

2.2 Unigram Model

The unigram model assumes that the occurrence of each word in a sentence is independent of other words. This means that the occurrence of a word does not rely on the context of preceding or following words. Thus, the probability of a sentence can

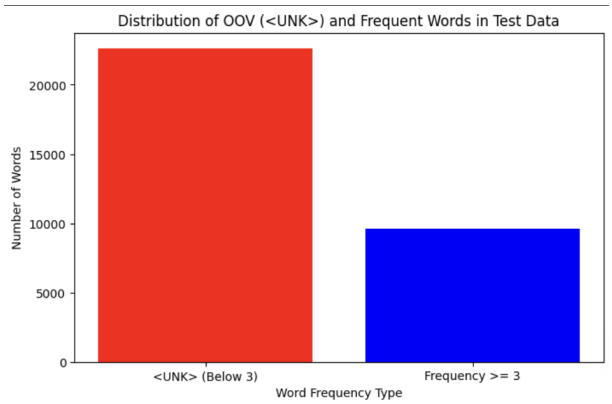

Figure 3: Distribution of OOV (<UNK>) and Frequent Words in Test Data

be calculated as the product of the probabilities of each word appearing independently. The unigram model is simple and computationally efficient, making it suitable for basic word frequency statistics. However, it ignores dependencies between words, so it cannot capture context or syntactic structures.

The probability of each word w_i in the unigram model can be expressed as:

$$P(w_i) = \frac{\text{Count}(w_i)}{\text{Total Words}}$$

2.3 Bigram Model

The bigram model considers the dependency between a word and the word immediately preceding it. Each word's probability is based on the occurrence of the previous word, making it more reflective of language structure compared to the unigram model. The bigram model can capture some common word pair relationships, but it only considers the preceding word, so it cannot capture longer contextual dependencies.

In the bigram model, the conditional probability of a word given its previous word w_{i-1} can be expressed as:

$$P(w_i|w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

2.4 Trigram Model

The trigram model considers the dependency between a word and the two preceding words. Each word's probability depends on the two previous words, making the model more accurate in reflecting language structure. For example, it can capture phrase sequences like "I am going," making it suitable for higher-level language analysis.

In the trigram model, the conditional probability of a word given its two preceding words w_{i-2} and w_{i-1} can be expressed as:

$$P(w_i|w_{i-2}, w_{i-1}) = \frac{\text{Count}(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})}$$

3 Perplexity

Perplexity is a commonly used evaluation metric in language models to measure the model's predictive ability on a given set of language data. The concept originates from information theory and can be intuitively understood as a measure of "uncertainty." formula for calculating perplexity is: Given a sequence of words $w_1, w_2, \ldots, w_M$

$$\text{Perplexity} = 2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i|\text{context})}$$

where:

- M is the total number of words in the sequence (including <STOP> tokens but excluding <START> tokens).
- $P(w_i|\text{context})$ represents the conditional probability of the i-th word w_i given its context.
- **Low Perplexity**: Indicates that the model is making accurate predictions for the language data.
- **High Perplexity**: Indicates that the model's predictions are less accurate.

In different n-gram models, perplexity varies based on the model's complexity:

- In a **Unigram model**, perplexity is generally higher because it only considers the independent probability of each word without any context. The perplexity for the unigram model is calculated as:

$$\text{Perplexity}_{\text{unigram}} = 2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i)}$$

  where M is the total number of tokens in the dataset (including <STOP> tokens but excluding <START> tokens), and $P(w_i)$ is the probability of word w_i in the unigram model.

- In a **Bigram model**, perplexity tends to be lower than in the unigram model because it takes into account the dependency on the preceding word. The perplexity for the bigram model is calculated as:

$$\text{Perplexity}_{\text{bigram}} = 2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i|w_{i-1})}$$

  where $P(w_i|w_{i-1})$ represents the probability of word w_i given the previous word w_{i-1}, calculated based on the frequency of word pairs in the dataset. Here, M again denotes the total number of tokens.

- In a **Trigram model**, perplexity is often even lower because it further considers dependencies on the previous two words. The perplexity for the trigram model is calculated as:

$$\text{Perplexity}_{\text{trigram}} = 2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 P(w_i|w_{i-2},w_{i-1})}$$

  where $P(w_i|w_{i-2}, w_{i-1})$ represents the probability of word w_i given the two previous words w_{i-2} and w_{i-1}. Like the previous models, M is the total number of tokens in the dataset.

3.1 Summary

In conclusion, each model's perplexity (unigram, bigram, and trigram) indicates how accurately it guesses the subsequent word in the sequence; the higher-order models are able to capture more contextual information. We may assess how well each model captures linguistic structure by comparing these perplexity scores between models.

Higher perplexity denotes more prediction difficulty, but lower perplexity shows that the model's predictions for the data are more accurate. We may evaluate how well each model captures linguistic structure by comparing perplexity across models.

3.2 Perplexity on Test Sentence

For the test sentence "HDTV .":

- **Unigram Model**: Perplexity = 658.04
- **Bigram Model**: Perplexity = 63.71
- **Trigram Model**: Perplexity = 39.48

These results confirm that the perplexity values obtained from my implementation are accurate. It also demonstrates that as the order of the n-gram model increases, the model better understands sentence structure, leading to reduced perplexity.

3.3 Perplexity Results on Training and Development Sets

Unigram Model: The Unigram model only considers the independent occurrence probability of each word without capturing dependencies between words, which limits its understanding of language structure.

- Training Set Perplexity: 976.54
- Development Set Perplexity: 630.49

Bigram Model: The perplexity significantly decreases, indicating that this model can effectively utilize the dependency on the previous word for predictions. However, the higher perplexity on the development set compared to the training set suggests possible overfitting, indicating that the model has limited generalization ability to new data.

- Training Set Perplexity: 77.07
- Development Set Perplexity: 341.64

Trigram Model: The Trigram model fits the training data exceptionally well, but its very high perplexity on the development set (6846.71) indicates poor adaptability to new data.

- Training Set Perplexity: 7.87
- Development Set Perplexity: 6846.71

4 Linear Interpolation Smoothing

To enhance the effectiveness of the language model, we implemented linear interpolation smoothing across the MLE unigram, bigram, and trigram models. The calculation formula is as follows:

$$\begin{aligned} P_{\text{smooth}}(w_j|w_{j-2}, w_{j-1}) = \lambda_1 P_{\text{MLE}}(w_j) \\ + \lambda_2 P_{\text{MLE}}(w_j|w_{j-1}) \\ + \lambda_3 P_{\text{MLE}}(w_j|w_{j-2}, w_{j-1}) \end{aligned}$$

where:

- P_{smooth} represents the smoothed probability.
- The hyperparameters $\lambda_1, \lambda_2, \lambda_3$ correspond to the weights on the unsmoothed unigram, bigram, and trigram models, respectively.
- The condition $\lambda_1 + \lambda_2 + \lambda_3 = 1$ ensures that the probabilities are normalized.

5 Experiments

We defined five sets of lambda values for experimentation:

- $(\lambda_1 = 0.3, \lambda_2 = 0.3, \lambda_3 = 0.4)$
- $(\lambda_1 = 0.1, \lambda_2 = 0.3, \lambda_3 = 0.6)$
- $(\lambda_1 = 0.2, \lambda_2 = 0.4, \lambda_3 = 0.4)$
- $(\lambda_1 = 0.5, \lambda_2 = 0.3, \lambda_3 = 0.2)$
- $(\lambda_1 = 0.4, \lambda_2 = 0.4, \lambda_3 = 0.2)$

Each set of lambda values was used to calculate the perplexity on the training and development sets. The results are as follows:

- **Lambda values** ($\lambda_1 = 0.3$, $\lambda_2 = 0.3$, $\lambda_3 = 0.4$) - Train Perplexity: 20.85, Development Perplexity: 336.65
- **Lambda values** ($\lambda_1 = 0.1$, $\lambda_2 = 0.3$, $\lambda_3 = 0.6$) - Train Perplexity: 17.17, Development Perplexity: 440.19
- **Lambda values** ($\lambda_1 = 0.2$, $\lambda_2 = 0.4$, $\lambda_3 = 0.4$) - Train Perplexity: 20.49, Development Perplexity: 342.69
- **Lambda values** ($\lambda_1 = 0.5$, $\lambda_2 = 0.3$, $\lambda_3 = 0.2$) - Train Perplexity: 31.09, Development Perplexity: 312.80
- **Lambda values** ($\lambda_1 = 0.4$, $\lambda_2 = 0.4$, $\lambda_3 = 0.2$) - Train Perplexity: 29.69, Development Perplexity: 302.60

The best lambda values and their corresponding development set perplexity are as follows:

- **Best Lambda values**: $\lambda_1 = 0.4$, $\lambda_2 = 0.4$, $\lambda_3 = 0.2$
- **Best Development Perplexity**: 302.60
- **Train Perplexity with Best Lambda**: 29.69
- **Test Perplexity with Best Lambda**: 302.32

This configuration balances the influence of the unigram and bigram models, while assigning a smaller weight to the trigram model. Reducing the weight of the trigram ($\lambda_3 = 0.2$) may enhance the model's generalization ability on the development and test sets by avoiding overfitting to high-order dependencies.

5.1 Automatic Search with Optuna

Using Optuna as a hyperparameter search tool allowed us to explore a wide range of lambda combinations, identifying the optimal values more precisely. This search yielded the following best configuration and corresponding perplexities:

- **Optuna Best Lambda values**: $\lambda_1 = 0.2169$, $\lambda_2 = 0.6390$, $\lambda_3 = 0.1441$
- **Best Development Perplexity**: 188.60
- **Train Perplexity with Best Lambda**: 25.48
- **Test Perplexity with Best Lambda**: 307.92

From Optuna's hyperparameter optimization, we observe a high weight on the bigram ($\lambda_2 = 0.6390$), indicating that two-word associations are the most significant features in this model. The reduced weights on the unigram and trigram components allow unigram frequency references to be retained, while limiting excessive reliance on high-order dependencies from the trigram. This adjustment reduces potential issues of data sparsity and improves the model's generalization capacity. The lambda values found through Optuna effectively balance the levels of dependency within the n-gram model, helping us determine the appropriate weighting of term frequency for this dataset.

5.2 Effect of Training Data Volume

We tested the model's performance using the full training data versus using only half of the training data. The results are as follows:

- **Using Full Training Data**:
  - Train Perplexity: 29.69
  - Development Perplexity: 302.60
- **Using Half of the Training Data**:
  - Train Perplexity: 29.67
  - Development Perplexity: 302.60

Here, it demonstrates that the model's generalization is strong and that the smaller dataset size has no discernible impact on its performance, given the identical perplexity levels with half of the data.

5.3 Effect of OOV Tagging Threshold

We experimented with changing the OOV tagging threshold by setting the threshold for marking words as <UNK> to words that appear less than 5 times, instead of less than 3 times. The results are as follows:

- **Using OOV Threshold of < 5 for <UNK> Tagging**:
  - Train Perplexity with <UNK> threshold 5: 26.68
  - Development Perplexity with <UNK> threshold 5: 152.59

When the OOV threshold was raised to five, the development perplexity dropped significantly to 152.59. Training confusion decreased from its prior values. This decrease suggests that treating more uncommon terms as <UNK> makes it easier for the algorithm to forecast the training data. Overall, these findings show that the model may achieve lower perplexity and improve performance on both training and development sets by employing a higher threshold for identifying words as <UNK>.